

E-COMMERCE SHOPPING SYSTEM

Technical Documentation and Implementation Guide

1. System Overview

The E-Commerce Shopping System (ECSS) is a console-based C++ application designed to simulate a modern online shopping platform. It provides users with a seamless interface to manage their shopping lifecycle from browsing to checkout.

Key Functional Capabilities:

- Product discovery and browsing
- Interactive shopping cart management
- Secure checkout process
- Historical order tracking
- Inventory stock management

2. Core Concepts Used

- **Object-Oriented Programming (OOP):** Structural implementation using Encapsulation for Product, Cart, Order, and User classes.
- **Static Members:** Centralized tracking for global metrics like total registered users and total orders placed.
- **File Handling:** Persistent storage of the product database and user order history using file streams.
- **Composition:** Design pattern where the Cart class contains and manages multiple Product objects.
- **Arrays/Vectors of Objects:** Systematic handling of product listings and cart contents.

- **Menu-Driven Navigation:** Logic controlled via switch statements for a user-friendly console experience.

3. System Architecture

```
E-Commerce System
|
├── Product Class
|   ├── Name, Price, Stock
|   └── Update Stock Logic
|
├── Cart Class (Composition)
|   ├── List of Products
|   └── Bill Calculation
|
├── Order Class
|   ├── Checkout Logic
|   └── History Management
|
└── User Class
    ├── Browse Products
    ├── Manage Cart
    └── Place Order
```

4. Class Descriptions

Product Class

Handles the fundamental unit of the inventory.

- **Attributes:** Product name, Price, Stock level.
- **Functions:** Display product details, decrement stock upon purchase.

Cart Class (Composition)

Manages the user's active selection before purchase.

- Features: Dynamic addition/removal of products, real-time total bill calculation.

Order Class

Manages the transition from cart to finalized transaction.

- Features: Finalizes checkout, saves data to permanent history files, generates purchase receipts.

User Class

The primary interface for interaction.

- Functions: Browsing the product catalog, interacting with the cart, and initiating the checkout flow.

5. C++ Source Code Implementation

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

class Product {
public:
    string name;
    double price;
    int stock;

    Product(string n, double p, int s) : name(n), price(p), stock(s) {}

    void display() {
        cout << name << " - $" << price << " (Stock: " << stock << ")" << endl;
    }
};

class Cart {
    vector<Product> items;
public:
```

```

void addItem(Product p) {
    items.push_back(p);
    cout << p.name << " added to cart." << endl;
}

double calculateTotal() {
    double total = 0;
    for (auto &item : items) total += item.price;
    return total;
}

};

class User {
public:
    Cart userCart;
    void browse(vector<Product>& products) {
        cout << "\n--- Available Products ---" << endl;
        for (int i = 0; i < products.size(); i++) {
            cout << i + 1 << ". ";
            products[i].display();
        }
    }
};

int main() {
    vector<Product> inventory = {
        Product("Laptop", 999.99, 10),
        Product("Mouse", 25.50, 50),
        Product("Keyboard", 45.00, 30)
    };

    User currentUser;
    int choice;

    cout << "Welcome to E-Commerce Console" << endl;
    currentUser.browse(inventory);

    cout << "\nEnter product number to add to cart (0 to exit): ";
    cin >> choice;

    if (choice > 0 && choice <= inventory.size()) {
        currentUser.userCart.addItem(inventory[choice-1]);
        cout << "Total Bill: $" << currentUser.userCart.calculateTotal() << endl;
    }

    return 0;
}

```

